

Reversa Bugs, Documento de Entendimento

Proposta para aprovação de Sandeco. Nenhuma implementação foi feita: este documento registra o que foi entendido.
Data: 2026-07-15 | Redação: Claude (Fable 5) | Segunda opinião: Codex (seção 12) | Parecer de prática diária incorporado (seção 13)
Origens: `evolucao/bugs/conversa_reversa_bugs_rastreabilidade.md` e `evolucao/bugs/parecer_reversa_bugs_pratica_diaria_para_claude.md`

1. Resumo executivo

O Time Reversa Bugs adiciona ao Reversa uma **memória causal de defeitos**: um registro de bugs em Markdown, dentro do projeto do usuário, projetado para ser lido e mantido por agentes LLM em qualquer engine (Claude Code, Codex, Cursor, Gemini CLI). Cada bug é um arquivo com front matter YAML, rastreável às specs extraídas pelo Reversa (`_reversa_sdd/`), ao código e aos testes. O time tem 5 comandos e uma regra central herdada da conversa original: **documentar bug e corrigir bug são atos brutalmente separados**.

Comando	Papel	Escreve onde
<code>/reversa-bug</code>	Registra e rastreia um bug reportado. Nunca corrige.	<code>_reversa_bugs/BUG-XXX-slug/</code> (a pasta única do bug)
<code>/reversa-bug-fix</code>	Reproduz, acha a causa raiz, cria teste de regressão, corrige com diff aprovado e dá o veredito de spec.	código (com aprovação), a pasta do bug (<code>fix/</code>), <code>_reversa_sdd/addenda/</code>
<code>/reversa-bug-debate</code>	Debate multiagente (épocas fixas + juiz isolado) sobre a estratégia de correção. Sempre opt-in.	<code>debate/</code> dentro da pasta do bug
<code>/reversa-depth-inspection</code>	Pente-fino de uma feature problemática com lentes especializadas. Só diagnostica.	<code>_reversa_bugs/inspections/</code> + pastas de bugs novos
<code>/reversa-bug-graph</code>	Regenera as views: índice, matriz BUG x BUG, grafo, impact score e a matriz de rastreabilidade BUG ↔ SPEC (nas duas pontas).	<code>_reversa_bugs/*.md</code> + espelho <code>_reversa_sdd/traceability/bugs.md</code>

2. Origem e decisões já tomadas

O design nasce da conversa Sandeco x ChatGPT (bugs como nós de rastreabilidade SPEC ↔ CODE ↔ TEST ↔ BUG) e foi refinado nesta sessão com as seguintes decisões de Sandeco:

Decisões travadas

- **5 comandos:** `/reversa-bug` , `/reversa-bug-fix` , `/reversa-depth-inspection` , `/reversa-bug-graph` , `/reversa-bug-debate` .
- **Registro canônico em `_reversa_bugs/`** na raiz, seguindo o padrão das pastas do Reversa e a diretiva `non-destructive`.
- **Pente-fino só diagnóstica:** cada achado vira um `BUG-XXX.md` ; a correção é sempre um ato posterior e separado.
- **Debate é opt-in explícito:** o Reversa avisa que a opção existe, explica que demora ($N \text{ agentes} \times R \text{ rodadas} + 1 \text{ juiz}$) e só roda se o usuário aceitar.
- **Debatedores multi-harness** (Codex, OpenCode, Gemini CLI): o Reversa **avisa** quando detecta outros harness instalados, mas só os inclui no debate se o usuário **aceitar ou pedir explicitamente**.
- **Diff e handoff:** toda correção mostra o diff antes de aplicar; toda etapa termina com handoff manual ("Digite CONTINUAR"). Nada avança sozinho.
- **Veredito de spec:** toda correção verifica se a spec original do código com bug deve ser versionada e alterada, e os diffs (código e spec) ficam registrados juntos no bug.
- **Pasta única por bug:** todos os documentos de um bug (registro, evidências, debate, diffs da correção) vivem numa única pasta `BUG-XXX-slug/` . Nada daquele bug fica espalhado em pastas globais. (O layout físico do status tem duas variantes, ver seção 4; a recomendação do Codex é pasta plana.)
- **Matriz de rastreabilidade BUG ↔ SPEC:** cruzamento explícito, nas duas direções, entre a pasta do bug e a seção de spec em `_reversa_sdd/` que define o comportamento. O lado da spec também registra os bugs que a atingem (via arquivo espelho gerado, já que a spec original nunca é editada).
- **Entrega:** spec consolidada em `specs/reversa-bugs/` + implementação + registro no installer + bump para 1.2.52. **Somente após autorização.**
- **Parecer de prática diária incorporado** (2026-07-15): o bug deixa de modelar só "consertar código" e passa a modelar o ciclo de vida do defeito. Triagem completa na seção 13: o que entrou integral, o que entrou simplificado, o que foi adiado.

3. Princípios do Reversa que o time herda

- **Non-destructive:** nenhum artefato pré-existente do projeto é apagado ou sobrescrito. O time só cria arquivos em `_reversa_bugs/` e adendos em `_reversa_sdd/addenda/` . A exceção controlada é o próprio ato de corrigir código no `/reversa-bug-fix` , que segue o precedente do `/reversa-coding` : só com diff exibido e aprovação explícita.
- **Multi-engine:** nada depende de recurso exclusivo de um harness. Onde houver subagentes (debate, pente-fino), existe fallback sequencial documentado.
- **Handoff manual:** cada agente termina sugerindo o próximo passo e aguardando "CONTINUAR".
- **Menus padrão Reversa:** opções com label + descrição + opção final "Outro" aberta.
- **Vendor-in:** o loop de debate multiagente é copiado e adaptado para dentro do Reversa, sem depender de skills globais da máquina de Sandeco.
- **Source of truth:** o arquivo `BUG-XXX.md` é a verdade. Índice, matriz e grafo são views regeneráveis.

4. A pasta `_reversa_bugs/` : uma pasta por bug

Cada bug é uma pasta, e tudo daquele bug mora ali dentro. Nenhum artefato de um bug fica em pasta global: olhando `BUG-007-desconto-duplicado/` você vê o registro, as evidências, o debate (se houve) e a correção, sem caçar nada.

```
_reversa_bugs/
├─ README.md           regras do registro, lifecycle, closure policy do projeto (gerado na 1ª execução)
├─ taxonomy.yaml       vocabulário controlado de area/module/feature (semeado da extração _reversa_sdd/)
├─ bugs/
│  └─ BUG-20260715-A7K3-desconto-duplicado/ ← a pasta única do bug
│     ├── bug.md        registro canônico (front matter YAML com status + phase, source of truth)
│     ├── evidence/     logs, prints, dumps, cápsula de reprodução (nunca logs gigantes no Markdown)
│     ├── debate/       se aberto: rodadas, convergência, resposta-final.md
│     └── fix/          change set: diffs de código, spec, config, scripts de reparo, verification.md
├─ BUG-20260716-P9M2-.../ ...
├─ inspections//       relatórios do pente-fino (apontam para os bugs por ID)
└─ generated/          views regeneráveis, nunca editadas à mão
   ├── index.md         resumo e tabelas por status/phase + caminho atual de cada bug
   ├── catalog.jsonl    catálogo compacto (front matter normalizado) para busca em duas etapas
   ├── matrix.md        relações BUG x BUG (lista esparsa de arestas)
   ├── graph.md         grafo de relações + clusters + impact score
   └── spec-matrix.md   matriz de rastreabilidade BUG ↔ SPEC (seção 6.6)
```

Identidade merge-safe: o ID canônico é `BUG-<data>-<sufixo>` (ex.: `BUG-20260715-A7K3`), globalmente único mesmo com dois harness registrando bugs ao mesmo tempo em worktrees diferentes (Claude num, Codex noutro). Um `display_number` humano opcional (42) vive no front matter para conversa do dia a dia.

Onde vive o status: decisão fechada pela dupla revisão

Os dois pareceres independentes (Codex, seção 12.7; prática diária, seção 13) convergiram: **a pasta do bug nunca se move; status e phase vivem no front matter e as views materializam as visões por estado**. Manter pasta e campo dizendo a mesma coisa é redundância que exige validação de divergência. A tabela abaixo fica como registro da comparação:

	A. Pastas de status (conversa original)	B. Pasta plana + status no front matter (recomendação firme do Codex)
Layout	<code>open/BUG-007/</code> , <code>active/BUG-007/</code> , <code>resolved/BUG-007/</code> : a pasta inteira do bug se move a cada transição.	<code>_reversa_bugs/BUG-007/</code> fixa para sempre; <code>status</code> : vive só no <code>bug.md</code> ; o <code>index.md</code> apresenta as visões por status.
Prós	Status visível no explorador de arquivos, sem abrir nada.	Endereço físico imutável: links nunca quebram, sem conflito de rename entre branches no git, ferramentas acham o bug num lugar só.
Contras	Todo link externo quebra a cada transição; duas branches movendo o mesmo bug geram conflito rename/rename; buscar um ID exige olhar 3 lugares.	Saber o status exige abrir o <code>bug.md</code> ou o <code>index.md</code> .

Como disse o Codex: "a organização por estado é uma projeção, não parte da identidade do bug". Referências entre documentos usam sempre o **ID**, nunca o caminho.

Ontologia: 3 status + phase

`status` segue com só 3 valores (`open` , `active` , `resolved`), e um campo `phase` separado registra a etapa do ciclo de vida dentro de `active` (ex.: `mitigating` , `reproducing` , `diagnosing` , `patching` , `observing` , `awaiting-human`). Bloqueio é uma **condição**, não um estado: campo estruturado `blocking`: (por outro bug ou por causa externa) com `is_blocked` derivado. Um bug só vira `resolved` quando a **closure policy do projeto** é satisfeita (seção 13): biblioteca local fecha com testes de regressão passando; serviço em produção só fecha depois de entregue e observado sem recorrência. E todo fechamento carrega um `resolution_kind` (`fixed` , `duplicate` , `invalid` , `cannot-reproduce` , `spec-only` , `instrumentation-required`).

taxonomy.yaml

Impede que agentes inventem nomes inconsistentes. Os campos `area` , `module` e `feature` de todo bug DEVEM usar valores deste catálogo. Se nada servir, o agente usa `unclassified` e propõe o novo termo em Agent Notes. Na primeira execução, o catálogo é semeado a partir dos módulos da extração (`_reversa_sdd/architecture.md`).

5. Anatomia de um bug (exemplo completo)

Exemplo em um sistema legado de vendas cuja extração do Reversa registrou a regra "desconto máximo de 10% por pedido" em `_reversa_sdd/domain.md` :

```

Arquivo: _reversa_bugs/bugs/BUG-20260715-A7K3-desconto-duplicado/bug.md
---
schema_version: 1
id: BUG-20260715-A7K3      # identidade canônica, merge-safe
display_number: 7          # apelido humano: "BUG-007" na conversa
title: Desconto aplicado duas vezes no fechamento do pedido
status: open               # open | active | resolved (a pasta nunca carrega o status)
phase: triaging            # etapa do ciclo de vida
severity: high
priority: P1
created: 2026-07-15
updated: 2026-07-15

origin:
  type: manual-report      # ou github-issue, ci-failure, telemetry, inspection...

area: vendas
module: checkout
feature: desconto

labels:
  - financeiro

visibility: normal         # normal | internal | restricted (fluxo de segurança)

reproduction:
  classification: deterministic # deterministic | intermittent | environment-dependent | not-reproduced
  rate: 10/10

blocking: []              # condições que travam o bug (outro bug ou causa externa)

relationships:
  - bug: BUG-20260701-Q2R8  # aresta canônica, gravada uma vez; inversa é derivada
    type: related-to
    state: proposed         # proposed | supported | confirmed | rejected

traceability:
  specs:
    - _reversa_sdd/domain.md#regras-de-desconto
  affected_code:
    - src/checkout/fechamento.py
  root_cause: null         # preenchido pelo fix, com estado epistemológico e evidências
  reproduction_tests: []   # prova que o defeito relatado aparece
  regression_tests: []     # protege o comportamento que não pode voltar a quebrar

spec_verdict: null        # preenchido pelo fix, com aprovação humana
change_set: []            # mudanças corretivas tipadas (test, code, config, data-repair, spec...)

closure:
  policy: local-software   # detectada na 1ª execução do time no projeto
  satisfied: false
  resolution_kind: null    # fixed | duplicate | invalid | cannot-reproduce | spec-only | instrumentation-required
---

# Desconto aplicado duas vezes no fechamento do pedido

## Summary
Pedidos com cupom têm o desconto aplicado no carrinho e novamente no fechamento,
resultando em desconto efetivo acima do teto de 10% definido na spec.

## Expected Behavior
Conforme `_reversa_sdd/domain.md#regras-de-desconto`: desconto máximo de 10%
por pedido, aplicado uma única vez.

```

Actual Behavior

Pedido de R\$ 100 com cupom de 10% fecha em R\$ 81 (desconto de 19%).

Steps to Reproduce

1. Adicionar item de R\$ 100 ao carrinho
2. Aplicar cupom DEZ10
3. Fechar o pedido
4. Conferir o total: R\$ 81 em vez de R\$ 90

Evidence

- Log: evidence/fechamento.log (caminhos relativos à pasta do bug)
- Print: evidence/total-errado.png

Suspected Area

`src/checkout/fechamento.py`, função `aplicar\_ajustes()`.

Acceptance Criteria

- Desconto aplicado exatamente uma vez, teto de 10% respeitado
- Teste de regressão cobrindo pedido com cupom
- Testes existentes continuam passando

Agent Notes

Não alterar o cálculo de frete durante a correção.

Por que o front matter importa

Ele transforma o registro num banco de dados navegável por grep. O usuário pode pedir "corrija todos os P1 abertos" ou "analise os bugs do módulo checkout" e o agente filtra por `priority: P1 + status: open ou module: checkout`, sem precisar ler tudo.

6. Os 5 comandos em detalhe

6.1 `/reversa-bug` , o registrador

1. Entrevista o usuário sobre o erro (o que esperava, o que aconteceu, como reproduzir), no padrão de menus do Reversa.
2. Procura **duplicata** nos bugs existentes; se achar, oferece atualizar o bug antigo em vez de criar outro.
3. Classifica com o `taxonomy.yaml` (area, module, feature) e define severity/priority com o usuário.
4. **Rastreia** (papel Tracer, interno): localiza a seção de spec em `_reversa_sdd/` que define o comportamento esperado, o código suspeito e os testes existentes. Sem spec correspondente, marca `spec-gap`.
5. **Correlaciona** (papel Correlator, interno): compara com os bugs existentes e propõe relações tipadas (`causes` , `blocked-by` , `duplicate-of` , `regression-of` ...).
6. Registra a **origem** (relato manual, issue, falha de CI, telemetria) e levanta **suspeita de segurança**: se houver indício (bypass de autenticação, exposição de segredo...), marca `security_suspected` , restringe a visibilidade e não expõe detalhe explorável em views nem a harness externos.
7. Cria a pasta `bugs/BUG-<data>-<sufixo>-slug/` com o `bug.md` (`status: open`) e as evidências em `evidence/` dentro dela.
8. Atualiza as views (index, matrix, graph) e **não corrige nada**.

Exemplo de fim de execução:

```
BUG-007 (BUG-20260715-A7K3) registrado em _reversa_bugs/bugs/BUG-20260715-A7K3-desconto-duplicado/  
Spec vinculada: domain.md#regras-de-desconto  
Relação proposta (proposed): related-to BUG-003 (arredondamento no mesmo módulo)  
  
Digite CONTINUAR para prosseguir com /reversa-bug-fix BUG-007,  
ou registre outro bug com /reversa-bug.  
(Os comandos aceitam o ID canônico ou o número humano.)
```

6.2 `/reversa-bug-fix` , o orquestrador do ciclo de vida

É o comando que leva o defeito da triagem até o fechamento comprovado. Nem todo projeto passa por todas as etapas: a closure policy e o contexto definem o caminho (um bug simples local segue reto; um incidente de produção passa por mitigação e observação).

1. Chamado com um ID (`/reversa-bug-fix BUG-007`). Sem ID, calcula o impact score e **sugere** qual corrigir (papel Prioritizer, interno).
2. **Mitigação antes de investigação, quando o dano é corrente** (severidade alta em sistema em uso): oferece via menu reduzir o estrago primeiro (desligar a funcionalidade, rollback, workaround), registra em `mitigation:` e distingue `MITIGATED` de `FIXED` . O bug segue `active` mesmo com o serviço restaurado.
3. Marca `status: active` e **reproduz** o defeito, gravando a **cápsula de reprodução** em `evidence/` (commit base, ambiente, comando executado, taxa de reprodução). Bug intermitente é cidadão de primeira classe: se não reproduzir, o fluxo pode concluir `resolution_kind: instrumentation-required` (adicionar log, métrica ou trace para capturar a próxima ocorrência é correção válida).
4. Investiga a **causa raiz com estado epistemológico**: hipótese, caminho causal e evidências, com estado `hypothesized` → `supported` → `confirmed` . Hipótese nunca entra no grafo como fato. Para regressão com commit bom e ruim conhecidos, usa `git bisect` formalmente e liga o bug ao commit culpado.

- 5. Avalia o **risco da mudança** (`change_risk` : baixo/médio/alto, com motivos: blast radius, contrato externo, dados, concorrência). Impacto do bug e risco da correção são coisas diferentes, e juntos definem a política: risco baixo + causa confirmada segue direto; incerteza diagnóstica ou risco alto puxa o debate.
- 6. Apresenta o **menu de estratégia** (quadro abaixo): correção direta ou debate multiagente (modo `diagnosis` ou `repair` , seção 6.3).
- 7. **Gate 1, o teste**: cria o **teste de reprodução** (prova que o defeito relatado aparece) e o(s) **teste(s) de regressão** (protegem o que não pode voltar a quebrar; são coisas distintas). Mostra o diff dos testes, aguarda aprovação, aplica e demonstra que **falham**.
- 8. **Gate 2, o change set**: monta o **Correction Change Set**, o conjunto tipado de mudanças corretivas (código, config, migration, reparo de dados, spec...), porque nem todo bug se resolve com patch de código. Mostra todos os diffs, aguarda aprovação, aplica e demonstra que os testes **passam**.
- 9. Avalia o **impacto em dados**: código corrigido não conserta estado histórico corrompido (`code healed ≠ system healed`). Se há registros antigos errados, o reparo de dados entra no change set com dry-run e rollback.
- 10. Executa o **veredito de spec** (seção 8), recomendando com evidências; **a decisão é humana**.
- 11. Fecha conforme a **closure policy**: preenche a Resolution com o change set e os diffs juntos em `fix/` , marca o `resolution_kind` e atualiza as views (incluindo a matriz BUG ↔ SPEC das duas pontas). Biblioteca local fecha aqui; serviço em produção fica em `phase: observing` até a janela de observação confirmar a não recorrência. `resolved: fixed` exige causa confirmada, testes de regressão e veredito de spec.

```
Menu do passo 4 (padrão Reversa):

Encontrei a causa raiz provável em src/checkout/fechamento.py:42.
Como você quer decidir a estratégia de correção?

[1] Correção direta
    Sigo com a minha hipótese: remover a reaplicação do cupom em
    aplicar_ajustes(). Mais rápido.

[2] Debate multiagente
    Abro um debate com N agentes por R rodadas + 1 juiz para comparar
    estratégias. Atenção: demora mais e custa mais, porque chama
    N x R subagentes (padrão 3 x 2 = 6 chamadas + juiz).
    Detectei Codex CLI instalado: se você aceitar, ele pode entrar
    como debatedor externo.

[3] Outro
    Descreva como você prefere decidir.
```

6.3 /reversa-bug-debate , a mesa de discussão

Vendor-in do loop de debate multiagente (épocas fixas), adaptado ao contexto de bugs, com **três modos**:

Modo	Quando	O que os debatedores disputam
<code>diagnosis</code>	Múltiplas hipóteses causais concorrentes.	Qual hipótese as evidências sustentam e quais probes discriminam entre elas.
<code>repair</code>	Causa confirmada, estratégias de correção concorrentes.	Menor mudança coerente, menor risco de regressão, melhor reversibilidade.
<code>spec</code>	Código, testes e spec divergem e não está claro quem manda.	Comportamento observado vs spec efetiva vs contratos; termina em recomendação , a decisão de spec é sempre humana.

- **Entradas travadas no início**: N agentes (padrão 3), R rodadas (padrão 2) e o problema P (a estratégia de correção do BUG-XXX, montado a partir do arquivo do bug + causa raiz + spec vinculada).

- **Época 0:** cada debatedor produz sua proposta de correção de forma independente, sem ver os outros.
- **Rodadas 1..R:** atualização síncrona por snapshot: cada debatedor lê as propostas de TODOS os outros da rodada anterior, critica e reescreve a própria. Ninguém lê atualização no meio da rodada.
- **Juiz isolado:** ao fim da rodada R, um juiz que não participou do debate e não vê o histórico de raciocínio (só as N propostas finais) aplica rubrica congelada (corrige a causa raiz? menor mudança coerente? risco de regressão? respeita a spec e os Agent Notes?) e sintetiza a estratégia vencedora em `resposta-final.md`.
- **Custo avisado antes:** o comando mostra a estimativa ($N \times R$ chamadas + 1 juiz) e o aviso de demora, e só roda com aceite.
- **Multi-harness:** se detectar outros harness CLI instalados (Codex, Gemini CLI, OpenCode), **avisa** que podem ocupar cadeiras de debatedor ou emitir avaliação, e só os usa com aceite ou pedido explícito. Debatedor externo é chamado em modo não-interativo e sua resposta é gravada no mesmo formato de arquivo dos demais.
- **Fallback multi-engine:** em harness sem subagentes, os N debatedores são executados sequencialmente pelo próprio agente, mantendo o protocolo de snapshot.

Estado em disco (auditável depois), dentro da pasta do próprio bug:

```
_reversa_bugs/bugs/BUG-20260715-A7K3-desconto-duplicado/debate/
├─ problema.md          P, N, R travados + rubrica do juiz
├─ rodada-0/agente-1.md ... agente-3.md
├─ rodada-1/agente-1.md ... agente-3.md
├─ rodada-2/agente-1.md ... agente-3.md  (agente-2 pode ser o Codex, se aceite)
├─ convergencia.md      quão próximas ficaram as propostas por rodada
└─ resposta-final.md    síntese do juiz, consumida pelo /reversa-bug-fix
```

6.4 /reversa-depth-inspection , o pente-fino

Para quando uma feature "vive dando problema" e o usuário quer uma varredura profunda, não um bug pontual.

- 1. Usuário indica a feature (ex.: "checkout"). O agente monta o **mapa da feature**: seções de spec, arquivos de código, testes e dados envolvidos.
- 2. Dispara **lentes especializadas**, como subagentes paralelos quando o harness suporta, senão em sequência:

Lente	O que procura
Conformidade com spec	Divergências entre o comportamento implementado e o que a extração diz que deveria acontecer.
Fluxo de dados	Valores que nascem, se transformam e persistem errado (nulos, arredondamentos, encoding, timezone).
Contratos e integrações	Chamadas externas, APIs e filas com contrato violado ou sem tratamento de falha.
Estados de erro e edge cases	Caminhos infelizes: entradas vazias, concorrência, limites, permissões.
Cobertura de testes	Regras da spec sem teste correspondente; testes que passam mas não provam nada.
Regressão histórica	git log da área: correções que voltaram, hotfixes recorrentes, arquivos que concentram mudanças.

- 3. Cada achado passa por **dedupe** contra os bugs existentes e, confirmado, vira um `BUG-XXX.md` registrado pelo protocolo do `/reversa-bug` , com severidade, rastreabilidade e relações.
- 4. Relatório consolidado em `_reversa_bugs/inspections/checkout/report.md` : mapa da feature, achados por lente, bugs criados, clusters suspeitos.
- 5. **Não corrige nada**. Handoff final sugere `/reversa-bug-fix` começando pelo bug de maior impacto.

6.5 /reversa-bug-graph , as views e a matriz BUG ↔ SPEC

Regenera as views a partir dos `bug.md` (que são a única fonte de verdade):

```
index.md (resumo + tabela)      matrix.md (relações)
| Status   | Qtde |      |      | BUG-003 | BUG-007 |
|-----|-----|      |      |      |      |
| Open     | 5    |      |      |      |      |
| Active   | 1    |      |      |      |      |
| Resolved | 12   |      |      |      |      |

graph.md (mermaid + clusters):
  BUG-012 -->|causes| BUG-031
  BUG-012 -->|causes| BUG-044
  BUG-018 -->|blocks| BUG-021

Impact score por bug aberto:
  impact = causados*3 + bloqueados*2 + regressoes*4 + relacionados
```

O grafo permite ler **clusters**: quatro bugs distintos convergindo no mesmo componente indicam causa estrutural comum, e o agente passa a recomendar "corrija BUG-012 primeiro, os outros três provavelmente caem junto". Isso alimenta o modo sem argumento do `/reversa-bug-fix`.

6.6 A matriz de rastreabilidade BUG ↔ SPEC (as duas pontas)

Todo bug nasce vinculado à seção de spec em `_reversa_sdd/` que definiu o comportamento (quem "criou" aquele código). Esse cruzamento vira uma matriz mantida **nas duas pontas**:

Ponta dos bugs: `_reversa_bugs/spec-matrix.md`

Spec (seção)	Bugs abertos	Bugs ativos	Bugs resolvidos
<code>_reversa_sdd/domain.md#regras-de-desconto</code>	BUG-007		BUG-003
<code>_reversa_sdd/architecture.md#checkout</code>	BUG-007, BUG-014		
(spec-gap, sem spec correspondente)	BUG-019		

Ponta das specs: `_reversa_sdd/traceability/bugs.md` (arquivo espelho, gerado)

<!-- GENERATED, DO NOT EDIT: regenerado por /reversa-bug-graph -->

domain.md#regras-de-desconto

- BUG-007 (open, P1): desconto aplicado duas vezes no fechamento

- BUG-003 (resolved/fixed): arredondamento incorreto, corrigido em 2026-07-02

- A spec original (`domain.md` , `architecture.md` , specs SDD) **nunca é editada**: o registro do lado da spec é o arquivo espelho `_reversa_sdd/traceability/bugs.md` , regenerável, ao lado do `traceability/code-spec-matrix.md` que a extração do Reversa já gera (sugestão do Codex, encaixa na convenção existente). Quando o conteúdo da spec precisa mudar (veredito `spec-desatualizada`), aí entra o adendo em `_reversa_sdd/addenda/` .
- Quem lê a spec descobre na hora quais bugs a atingem; quem lê o bug chega à spec pela `traceability.specs` do front matter. Um bug com `spec-gap` aparece na matriz na linha própria, denunciando comportamento sem especificação.
- As duas pontas são views regeneradas pelo `/reversa-bug-graph` e atualizadas automaticamente pelos demais comandos ao registrar ou resolver bugs.

7. Rastreabilidade em 3 dimensões

VERTICAL

HORIZONTAL

TEMPORAL

SPEC ↔ CODE ↔ TEST ↔ BUG

BUG ↔ BUG

BUG → FIX → REGRESSION

(o quadrado que fecha um bug)

(relações tipadas: causes, blocks, duplicate-of, regression-of...)

(o que foi corrigido precisa continuar verdadeiro)

CODE

SPEC

BUG

TEST

SPEC define. Código implementa. Bug denuncia a divergência.

Teste prova que a divergência foi eliminada.

A dimensão vertical é materializada na matriz BUG ↔ SPEC da seção 6.6, mantida nas duas pontas (`_reversa_bugs/spec-matrix.md` e `_reversa_sdd/bugs.md`). Regra dura no `README.md` do registro: um bug NÃO pode ser marcado `resolved` com

`traceability.specs` , `root_cause_code` ou `regression_tests` vazios. Se não existe spec para o comportamento, o bug carrega `spec-gap` e a lacuna precisa ser tratada antes da resolução.

A dimensão temporal conversa com o que o Reversa já tem: os testes de regressão de bugs podem ser vigiados pelo mesmo espírito do `regression-watch.md` do ciclo forward, e a re-extração futura (`/reversa`) enxerga os adendos de spec gerados pelas correções.

8. Veredito de spec: correções mapeadas e versionadas, com os diffs juntos

Pedido específico de Sandeco: **toda correção precisa verificar se a spec original do código com bug deve ser versionada e alterada, e os diffs andam juntos**. Ao final de cada correção o agente responde: "o que estava errado, o código ou a spec?"

Veredito	Significado	Ação sobre a spec
spec-correta	A spec descrevia o comportamento certo; o código divergiu.	Nenhuma. O fix realinha o código à spec.
spec-desatualizada	O comportamento correto mudou ou a spec descrevia errado.	Gera adendo versionado em <code>_reversa_sdd/addenda/</code> com o diff da spec (trecho antes / trecho como deve ser lido agora). A spec original nunca é editada (non-destructive, mesmo padrão do <code>/reversa-sync</code>).
spec-gap	Não existia spec para o comportamento.	Gera adendo que especifica o comportamento pela primeira vez, ligado ao bug.

E a seção Resolution do bug guarda **os dois diffs lado a lado**:

```
## Resolution (dentro do bug.md, após o fix)

Root cause (confirmed): reaplicação do cupom em aplicar_ajustes(),
introduzida no commit a1b2c3 (localizado via git bisect).
Spec verdict: spec-correta, aprovado por Sandeco
(domain.md#regras-de-desconto já definia o teto de 10%).
Resolution kind: fixed. Closure policy local-software satisfeita.

### Correction Change Set
| ID      | Tipo | Artefato                                     | Propósito                |
|-----|-----|-----|-----|
| CHG-001 | test | tests/checkout/test_desconto_unico.py      | reproduzir/proteger     |
| CHG-002 | code | src/checkout/fechamento.py                | eliminar causa raiz     |

### Diff do código (fix/CHG-002.diff, na pasta do bug)
- total = aplicar_cupom(total, cupom) # cupom já aplicado no carrinho
+ # cupom aplicado uma única vez, no carrinho (domain.md#regras-de-desconto)

### Diff da spec
Nenhum. Veredito spec-correta: o código foi realinhado à spec vigente.

### Testes
Reprodução: test_cupom_nao_reaplica (falhava antes do CHG-002, passa depois)
Regressão: test_teto_desconto_10pct (protege a regra da spec)
```

Exemplo do outro caminho, quando o veredito é `spec-desatualizada` :

Arquivo novo: `_reversa_sdd/addenda/BUG-012-spec-update-teto-desconto.md`

Adendo de spec por correção de bug: BUG-012
Vigente desde 2026-07-15. Origem: `/reversa-bug-fix` BUG-012.

Seção afetada
`_reversa_sdd/domain.md#regras-de-desconto`

Diff da spec (como a seção deve ser lida a partir de agora)
- Desconto máximo de 10% por pedido.
+ Desconto máximo de 10% por pedido, exceto na campanha Black Friday,
+ quando o teto passa a 25% mediante flag de campanha ativa.

Diff do código correspondente
`_reversa_bugs/bugs/BUG-20260710-C4T1-teto-black-friday/fix/CHG-002.diff`
(os dois diffs são registrados juntos, no adendo e na pasta do bug)

Efeito no ciclo do Reversa

A próxima re-extração (`/reversa`) encontra os adendos e os marca como superados ao regenerar as specs, exatamente como já acontece com os adendos do `/reversa-sync` . A extração permanece intocada entre re-extrações, mas quem a lê enxerga o sistema como ele é hoje.

9. O que acontece em cada engine

Capacidade	Harness com subagentes (ex.: Claude Code)	Harness sem subagentes
Debate (N debatedores)	Subagentes paralelos por rodada	O agente executa os N papéis em sequência, mantendo o snapshot síncrono
Juiz do debate	Subagente isolado, sem histórico dos debatedores	Passo separado que lê apenas os arquivos finais das propostas
Pente-fino (6 lentes)	Lentes em paralelo	Lentes em sequência
Debatedores externos	Qualquer harness pode chamar outro via CLI não-interativa (ex.: <code>codex exec</code>), com aceite do usuário	

10. Escopo da implementação (autorizada em 2026-07-15)

A entrega é:

- `specs/reversa-bugs/` : requirements.md, design.md e tasks.md consolidando este documento.
- 5 novos agentes em `agents/` (SKILL.md + references com o template de bug, o template do README do registro e o protocolo do debate).
- Registro do time no installer (`lib/installer/prompts.js` + `lib/commands/install.js`) como time próprio, instalável e adicionável via `add-agent` .
- `package.json` : 1.2.51 → 1.2.52 (só patch).

5. Menção do fluxo de bugs na documentação existente onde já há lista de times.

Fora do escopo desta entrega

Alterações em agentes existentes; hooks novos em `templates/` ; integrações ativas com GitHub/Sentry; postmortems. O trabalho não commitado da v1.2.51 (reversa-sync) que já estava no working tree é preservado em commit separado, sem mistura.

11. Pontos decididos ao longo das revisões

- **Layout do status:** pasta plana com status no front matter, fechado pela convergência dos dois pareceres (seções 12.7 e 13).
- **Numeração:** ID canônico merge-safe `BUG-<data>-<sufixo>` + `display_number` humano; colisão entre worktrees eliminada por construção.
- **Bloqueio:** campo estruturado `blocking:` , nunca um quarto status (12.2).
- **Registro do lado da spec:** espelho gerado em `_reversa_sdd/traceability/bugs.md` (12.7).
- **Nome do pente-fino:** mantido `/reversa-depth-inspection` , conforme pedido original de Sandeco.
- **Modo de controle:** `gated` como padrão (seção 13): leitura, reprodução isolada e diagnóstico fluem sem CONTINUAR; gate obrigatório em tudo que toca o projeto (testes, change set, spec efetiva, harness externo, operação destrutiva).

12. Parecer do Codex (segunda opinião)

O design acima foi submetido ao Codex (GPT-5, sessão `019f66ee`), que leu a conversa original, o padrão de skills do Reversa e o installer. Veredito geral: "**a arquitetura conceitual é forte, sobretudo na separação entre registrar, diagnosticar, decidir e corrigir**", porém "um pouco superdimensionada nas capacidades avançadas e subdimensionada nas invariantes básicas do registro". O parecer completo está no log da sessão; abaixo, o essencial por pergunta.

12.1 Riscos e lacunas apontados

- **Schema formal do front matter, com `schema_version`** : sem campos obrigatórios, tipos e enums definidos, 5 skills em várias engines produzirão variações incompatíveis.
- **Autoridade pasta vs front matter:** se o arquivo está em `active/` com `status: open` , o regenerador deve **parar com erro explícito**, nunca consertar em silêncio.
- **Relações canônicas:** gravar a aresta uma única vez (`caused-by` , `blocked-by` , `duplicate-of` , `regression-of`) e derivar as inversas (`causes` , `blocks`) nas views. Proibir autorrelação, referência inexistente e ciclo de `duplicate-of` .
- **resolution_kind** : a regra "só resolve com root cause + regression test" vale para `fixed` , mas não cobre `duplicate` , `invalid` , `cannot-reproduce` e `spec-only` . Sem isso, bugs legítimos ficam eternamente abertos ou ganham rastreabilidade artificial.
- **Dois gates de aprovação no fix:** o teste de regressão também altera o projeto. Gate 1: diff do teste, aprovar, aplicar, provar que **falha**. Gate 2: diff da correção (+ adendo), aprovar, aplicar, provar que **passa**.
- **Registrador central de IDs:** na inspeção paralela, nenhuma lente cria `BUG-XXX` direto; lentes produzem achados, um registrador único faz dedupe e atribui IDs em série. Isso também responde à questão de colisão de numeração da seção 11.
- **Spec efetiva:** toda análise considera spec original + adendos vigentes, nunca só a original.
- **Nada de caminho fixo:** resolver `output_folder` via `.reversa/state.json` , não presumir `_reversa_sdd/` .

- **Debate não é caminho padrão:** oferecer principalmente quando houver causas concorrentes, risco alto, mudança transversal ou desacordo sobre a spec. Impact score é heurística de triagem, não verdade matemática.

12.2 Ontologia de status: 3 é melhor que 4

O Codex confirma os 3 status: bloqueio é condição ortogonal ao estágio (um bug pode estar `open` e bloqueado por decisão humana). Mas recomenda estruturar o bloqueio além de bug-para-bug:

```
blocking:
- kind: bug
  target: BUG-031
- kind: external
  reason: "Aguardando credenciais do fornecedor"
  since: 2026-07-15
```

`is_blocked` passa a ser derivado, nunca um quarto status. Isso resolve o primeiro ponto em aberto da seção 11.

12.3 Veredito de spec: adendo é o caminho certo, com 4 ajustes

- **Adendo como overlay:** a "spec efetiva" é a base mais os adendos vigentes, não uma nova versão física do arquivo.
- **Aprovação humana do veredito:** uma spec extraída do legado pode descrever fielmente um comportamento errado. O agente recomenda o veredito com evidências, quem bate o martelo é o usuário.
- **Distinguir alteração de criação:** `spec-desatualizada` gera adendo com delta de seção existente; `spec-gap` gera overlay aditivo com requisito novo, sem fingir que altera seção inexistente.
- **Versionar o evento:** um arquivo imutável por decisão aprovada (ex.: `bug-BUG-042-v001.md`) com alvo, vigência, evidências e aprovação, em vez de reescrever adendo anterior.

Alerta importante: a re-extração do Reversa hoje marca todos os adendos como superados. Para adendos de bug (sobretudo `spec-gap`), isso só deve acontecer após verificar que a nova extração **absorveu** o comportamento. Na Resolution, manter código e spec unidos transacionalmente, mas com resumo + hash + link para o diff completo em `evidence/`, sem colar diffs gigantes no Markdown.

12.4 Debatedores externos: protocolo endurecido

- **Probe antes de oferecer:** detectar o executável não basta; sondar modo não-interativo, autenticação e capacidade read-only. Sem read-only verificável, a CLI externa só avalia material fornecido, nunca recebe acesso mutável ao projeto.
- **Arquivo padronizado por debatedor e rodada:** front matter com `protocol_version`, engine, modelo, rodada, hash do snapshot, status e duração; corpo com seções fixas (hipóteses, causa raiz, estratégia, teste, impacto na spec, riscos, confiança).
- **Juiz protegido:** recebe as soluções finais anonimizadas, em ordem embaralhada, tratadas como dados não confiáveis (defesa contra prompt injection e contra favorecer engine pelo nome).
- **Falhas:** timeout duro (padrão 10 min), uma repetição só para falha de transporte, quórum mínimo de `max(2, ceil(2N/3))` para continuar automaticamente; sem quórum, menu para o usuário. Debatedor que falha gera arquivo com `status: timeout|error`, nunca é substituído em silêncio. Se o juiz falhar, preservar tudo e não inventar vencedor.
- **Papéis:** distinguir `solver` (propõe e concorre) de `critic` (só avalia). O aviso de custo mostra a conta real: solvers x rodadas + críticos x rodadas + juiz.

12.5 Pente-fino: 6 lentes, com uma promoção e uma nova

- As 6 lentes fazem sentido; **rebaixar "regressão histórica via git"** de lente confirmatória para fonte auxiliar (histórico raso não prova ausência de defeito).
- **Nova lente obrigatória: concorrência e consistência temporal** (transações, idempotência, retries, condições de corrida, cache, ordenação de eventos).
- **Lentes condicionais**, ativadas por sinais: segurança/autorização, desempenho, configuração/migrations/flags, observabilidade.
- **Critério de "achado confirmado"**: desvio observável entre esperado e real, ou prova estática com caminho causal completo. Dívida técnica e suspeita ficam no relatório como achados, com `finding_id`, confiança e `promoted_to`, sem virar bug automaticamente.

12.6 Escala (200+ bugs)

- Gerar um **catálogo compacto derivado** (`catalog.jsonl`, só front matter normalizado) e fazer recuperação em duas etapas: filtrar o catálogo, ler o corpo só dos 10-20 candidatos.
- **Matriz NxN global não escala** (200x200 = 40 mil células vazias): virar lista esparsa de arestas e matrizes filtradas por cluster/status. Grafo mostra clusters e maiores impactos, com subgrafos por componente.
- Views com `generated_at`, versão do schema e digest da entrada; geração determinística, sem delegar a montagem de 200 registros ao contexto da LLM.
- Impact score calculado só sobre arestas canônicas, com peso limitado para `related-to`; referência entre bugs sempre por ID (as views resolvem o caminho da pasta atual).

12.7 Segunda rodada: parecer sobre a pasta única por bug e a matriz BUG ↔ SPEC

Após a decisão de Sandeco de concentrar tudo de um bug numa pasta só, o Codex foi consultado de novo (sessão `019f6716`).

Resumo:

- **Pasta plana, recomendação firme**: `_reversa_bugs/BUG-007/` com endereço físico imutável e `status` só no front matter. Mover a pasta entre `open/active/resolved` quebra links externos a cada transição, gera conflito rename/rename entre branches no git e obriga ferramentas a procurar o ID em três lugares. "A organização por estado é uma projeção, não parte da identidade do bug." Se Sandeco preferir manter a movimentação, mitigações obrigatórias: referências só por ID, links internos relativos, o registrador resolve ID → caminho atual, validação rejeita ID duplicado e divergência pasta vs `status`, e o nome da pasta carrega só o ID (slug fica no YAML).
- **Espelho do lado da spec: ideia correta**, desde que explicitamente uma view gerada, nunca outra fonte de verdade. Endereço recomendado: `_reversa_sdd/traceability/bugs.md`, ao lado do `traceability/code-spec-matrix.md` que a extração já produz. Cabeçalho "GENERATED, DO NOT EDIT", uma seção por spec, links resolvidos por ID. Não criar um arquivo por spec (multiplicaria derivados e pareceria alteração da spec original) e não deixar o vínculo só do lado dos bugs (o espelho melhora a descoberta por quem navega pelas specs). Os adendos seguem sendo os únicos documentos autorais que complementam o contrato original.
- **Recomendações anteriores permanecem** e uma fica mais importante: com bugs autocontidos, o registrador central de IDs deve ser append-only e a validação precisa pegar duplicata no merge (contador sequencial sozinho não impede colisão entre branches). `catalog.jsonl` passa a incluir o caminho calculado de cada bug; `resolution_kind` deve indicar quando houve adendo de spec; os dois gates de aprovação seguem inalterados.

Síntese do Codex para a nova estrutura: **"bug autocontido, endereço físico imutável, estado no YAML e todas as matrizes e catálogos como projeções regeneráveis."**

12.8 Posição da redação sobre os pareceres

Recomendação

Acatar integralmente as duas rodadas. Nenhuma recomendação do Codex contradiz as decisões travadas por Sandeco; todas endurecem o design (schema versionado, `resolution_kind`, bloqueio estruturado, dois gates no fix, aprovação humana do veredito de spec, registrador central de IDs, protocolo de debatedor externo com quórum, lente de concorrência, views esparsas com catálogo, espelho em `_reversa_sdd/traceability/bugs.md`). Se aprovado, tudo isso entra na spec `specs/reversa-bugs/` antes da implementação. Restam a Sandeco as duas decisões da seção 11: o layout do status (pasta plana, recomendada, ou pastas de status) e o nome do comando de pente-fino.

13. Parecer da prática diária: triagem e o que muda

O terceiro parecer (`parecer_reversa_bugs_pratica_diaria_para_claude.md`) trouxe a crítica mais estrutural: o desenho modelava bem "agente conserta código", mas não o **ciclo de vida completo de um defeito** (intake → triage → mitigate → reproduce → diagnose → fix → deliver → observe → close). A tese central foi adotada como norte do projeto:

Tese do Reversa Bugs

O sistema mantém uma memória causal verificável do defeito, repository-native, e usa agentes especializados como workers efêmeros para investigar, decidir, corrigir e comprovar a recuperação do software. Os agentes não são a arquitetura; a arquitetura é estado, evidência, políticas e rastreabilidade.

13.1 Acatado integral

- **Pasta não é fonte de estado + phase separada de status** (§14), convergindo com o Codex. Estrutura final na seção 4, com pasta única por bug preservada (a proposta do parecer de voltar a evidence/ e debates/ globais foi rejeitada por contrariar decisão de Sandeco).
- **Teste de reprodução ≠ teste de regressão** (§7).
- **Estado epistemológico** em causa raiz e relações BUG↔BUG (§8, §9): `hypothesized/supported/confirmed/rejected` , com evidências; relação `proposed` não altera priorização automática.
- **IDs merge-safe** (§13): `BUG-<data>-<sufixo> + display_number` .
- **Debate com 3 modos** (§10): `diagnosis` , `repair` , `spec` ; o modo `spec` termina em recomendação, decisão humana.
- **Correction Change Set** (§4): mudanças corretivas tipadas; um bug não produz necessariamente um patch de código.
- **code healed ≠ system healed** (§5): impacto em dados e reparo de estado histórico entram no change set.
- **Mitigação separada de correção** (§3): oferecida via menu quando o dano é corrente; `MITIGATED ≠ FIXED ≠ RESOLVED` .
- **Bugs intermitentes + instrumentation-required** (§18) e **git bisect formal** (§19).
- **Fluxo de segurança** (§23): campo `visibility` , sem detalhe explorável em views nem envio a harness externo sem aprovação.
- **Manter os 5 comandos** (§28): o ganho entra na máquina de estados e nos schemas, não em comandos novos.

13.2 Acatado simplificado (v1 enxuta)

- **Closure policy por perfil de projeto** (§2, §16): 3 perfis detectados na primeira execução (`local-software` , `package` , `production-service`), definindo o que "resolved" exige. Fases reduzidas ao núcleo útil.
- **Cápsula de reprodução** (§6): versão essencial (commit base, ambiente, comando, taxa), sem digests exaustivos.
- **Blocos opcionais por contexto**: delivery/PR/CI (§15), observação pós-fix (§17), versões e backports (§20), ownership via CODEOWNERS (§21), origem externa (§22). O schema comporta; o agente só preenche quando o projeto tem o recurso.
- **change_risk qualitativo** (§24): baixo/médio/alto com motivos, alimentando o menu de estratégia; sem score numérico de aparência exata.
- **Modos de controle** (§25): `gated` padrão; `supervised` e `autonomous` disponíveis. Mesmo em `autonomous`, `spec`, segurança e operações destrutivas continuam com gate.
- **IDs estáveis de spec** (§11): começa com ID + locator resolvido pelas views; catálogo completo fica para quando a re-extração provar necessidade.

13.3 Adiado deliberadamente

- **Postmortem policy** (§26): conversa com o futuro time de manutenção (SRE/DORA) já rascunhado em `specs/time-de-manutencao/`.
- **Integrações profundas** com GitHub/GitLab/Sentry: o schema registra referências, mas nenhuma integração ativa na v1.
- **Confidences numéricas** (`0.94`): substituídas por níveis qualitativos.

13.4 Régua de aceitação

Os 10 cenários do parecer (§30) viram critérios de aceitação da spec: bug simples local fecha sem burocracia; incidente de produção não fecha com teste local; corrupção de dados distingue código curado de sistema curado; intermitente permite instrumentação sem inventar causa; regressão liga bug ao commit culpado via bisect; múltiplas versões respeitam backport pendente; divergência de spec exige decisão humana; vulnerabilidade não vaza em views; dois harness em worktrees não colidem IDs; correção de alto risco considera `change_risk`.

Reversa Bugs, documento de entendimento. Gerado em 2026-07-15 na sessão de design com Sandeco, consolidando a conversa original, dois pareceres do Codex e o parecer de prática diária. Implementação autorizada por Sandeco em 2026-07-15.